

Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

The purpose of the TastyBite Restaurant App is to provide a simple mobile application where customers can browse food, search menu items, add food to cart, place orders, reserve tables, track orders, and manage their profile.

Managers can view orders and update their status through the Manager Dashboard.

1.2 Scope

TastyBite is a **frontend-only React Native mobile application**.

The application provides:

- Customer login and signup
- Manager login
- Food menu
- Food categories
- Search
- Shopping cart
- Checkout
- Order tracking
- Table reservation
- Customer profile
- About Us
- Manager dashboard
- Order status management
- Local/mock data and AsyncStorage for frontend persistence

2. Definitions and Acronyms

Term	Meaning
SRS	Software Requirements Specification
MVP	Minimum Viable Product
UI	User Interface
UX	User Experience
RN	React Native
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
FR	Functional Requirement
NFR	Non-Functional Requirement
DB	Database
AsyncStorage	Local storage for app data
Customer	Person who orders food
Manager	Person who manages restaurant orders
Menu	List of available food items

3. Overall Description

3.1 Product Perspective

TastyBite is a mobile restaurant application developed using React Native and Expo. It uses frontend mock data instead of a real backend server.

3.2 User Roles

Customer

A customer can:

- Login/signup

- View menu
- Search food
- Add food to cart
- Change cart quantity
- Checkout
- Track order
- Reserve a table
- View/edit profile
- View restaurant information

Manager

A manager can:

- Login
- View dashboard
- View orders
- Check order statistics
- Update order status
- Logout

4. User Stories

Customer User Stories

1. As a customer, I want to login so that I can access the application.
2. As a customer, I want to view the food menu so that I can select food.
3. As a customer, I want to search food so that I can quickly find an item.
4. As a customer, I want to filter food by category so that I can find food easily.
5. As a customer, I want to add food to my cart so that I can order it.

6. As a customer, I want to increase or decrease quantity so that I can control my order.
7. As a customer, I want to checkout so that I can place my order.
8. As a customer, I want to track my order so that I know its status.
9. As a customer, I want to reserve a table so that I can dine at the restaurant.
10. As a customer, I want to edit my profile so that my information is updated.

Manager User Stories

11. As a manager, I want to login so that I can access the dashboard.
12. As a manager, I want to view orders so that I can manage them.
13. As a manager, I want to update order status so that customers can see progress.
14. As a manager, I want to view order statistics so that I can monitor restaurant activity.

5. Functional Requirements

5.1 Authentication

FR-01: The system shall provide customer login.

FR-02: The system shall provide customer signup.

FR-03: The system shall provide manager login.

FR-04: The system shall validate login credentials.

5.2 Menu

FR-05: The system shall display available food items.

FR-06: The system shall display food name, price, category, rating, and image/emoji.

FR-07: The system shall allow customers to filter food by category.

FR-08: The system shall allow customers to add food to the cart.

5.3 Search

FR-09: The system shall provide a food search field.

FR-10: The system shall display matching food items based on the search text.

5.4 Cart

FR-11: The system shall display selected food items in the cart.

FR-12: The system shall allow customers to increase item quantity.

FR-13: The system shall allow customers to decrease item quantity.

FR-14: The system shall allow customers to remove items.

FR-15: The system shall calculate the subtotal and total price.

5.5 Checkout and Orders

FR-16: The system shall allow customers to enter name, phone number, and address.

FR-17: The system shall provide payment options.

FR-18: The system shall allow customers to place an order.

FR-19: The system shall display order tracking information.

FR-20: The system shall display order statuses such as Pending, Preparing, Ready, and Served.

FR-21: The system shall allow customers to cancel an order during tracking.

5.6 Reservation

FR-22: The system shall allow customers to enter reservation information.

FR-23: The system shall allow customers to select date and time.

FR-24: The system shall allow customers to select number of guests.

FR-25: The system shall allow customers to select a table.

FR-26: The system shall confirm a reservation after valid information is entered.

5.7 Profile

FR-27: The system shall display customer profile information.

FR-28: The system shall allow customers to edit profile information.

FR-29: The system shall provide a logout option.

5.8 Manager Dashboard

FR-30: The system shall display the manager dashboard.

FR-31: The system shall display order statistics.

FR-32: The system shall display recent orders.

FR-33: The system shall allow the manager to update order status.

FR-34: The system shall provide manager logout.

6. Non-Functional Requirements

6.1 Usability

The system should have a simple and user-friendly interface.

6.2 Performance

Screens should load quickly and user actions should respond without unnecessary delay.

6.3 Responsiveness

The application should work properly on different mobile screen sizes.

6.4 Maintainability

The code should be organized into screens, components, contexts, reducers, hooks, and data files.

6.5 Reliability

The application should handle invalid input and empty cart situations properly.

6.6 Frontend Only

The MVP does not require a real backend server or online database.

6.7 Data Persistence

The application can use AsyncStorage to store selected frontend data locally on the device.

7. Technology Stack

Technology	Purpose
React Native	Mobile application development
Expo	Development and testing
TypeScript	Application programming
Expo Router	Navigation
AsyncStorage	Local data storage
React Hooks	State and application logic
VS Code / PowerShell Development	

8. React Hooks Used

The application uses the following React concepts:

- `useState` — manages screen states and form fields.
- `useEffect` — performs side effects such as loading/saving data.
- `useRef` — manages references/timers.
- `useContext` — accesses shared cart data.
- `useReducer` — manages cart state.
- `useMemo` — calculates optimized values.
- `useCallback` — memoizes functions.
- Custom Hook — `useCart()` provides cart functionality.

9. Mock Data

The application uses frontend mock data for:

- Users
- Categories
- Menu Items
- Tables
- Reservations
- Orders

Example customer:

Email: customer@tastybite.com

Password: Tasty123

Example manager:

Email: manager@tastybite.com

Password: Manager123

10. Main Screens

Q1. Login / Signup

Purpose: Allows users to login or create an account.

UI: Email, password, account type, login/signup buttons.

Navigation: Login → Home or Manager Dashboard.

Q2. Home

Purpose: Main customer screen.

UI: Welcome message, offers, and quick action buttons.

Navigation: Menu, Cart, Reservation, Order Tracking, Profile, About.

Q3. Menu

Purpose: Allows customers to browse food.

UI: Search, categories, food cards, prices, Add button.

Navigation: Menu → Cart.

Q4. Cart

Purpose: Shows selected food.

UI: Food items, quantity controls, subtotal, delivery, total.

Navigation: Cart → Checkout.

Q5. Checkout

Purpose: Collects order information.

UI: Name, phone, address, payment option, order summary.

Navigation: Checkout → Order Tracking.

Q6. Order Tracking

Purpose: Shows order progress.

UI: Order number and order status.

Navigation: Order Tracking → Home/Menu.

Q7. Reservation

Purpose: Allows customers to reserve a table.

UI: Name, date, time, guests, table selection.

Navigation: Reservation → Home.

Q8. Profile

Purpose: Allows customers to view and edit their information.

UI: Name, email, phone, account options, logout.

Q9. About Us

Purpose: Provides restaurant information.

UI: Restaurant description, features, contact information.

Q10. Manager Dashboard

Purpose: Allows manager to manage restaurant orders.

UI: Statistics, recent orders, order status buttons.

Navigation: Manager → Order Management / Logout.

11. UML Diagrams

The following UML diagrams are required for the project:

11.1 Use Case Diagram

Shows interactions between:

- Customer
- Manager
- Login
- Browse Menu
- Search Food
- Manage Cart
- Place Order
- Track Order
- Reserve Table
- Manage Profile
- Manage Orders
- Update Order Status

11.2 Class Diagram

Main classes:

- User
- Customer
- Manager
- Category
- MenuItem
- Cart
- CartItem

- Order
- Reservation
- Table

Customer and Manager inherit from User.

11.3 Sequence Diagram

Shows the customer order process:

Login → Home → Menu → Add to Cart → Cart → Checkout → Place Order → Order Tracking

11.4 State Machine Diagram

Order states:

Pending → Preparing → Ready → Served

The order can also move to:

Cancelled

11.5 Component Diagram

Main components:

- React Native Screens
- Expo Router
- Cart Context
- Cart Reducer
- Custom Hooks
- AsyncStorage
- Mock Data

12. Limitations

- No real backend server.
- No real online payment system.
- No real restaurant database.

- No real-time order synchronization.
- Manager management cards are frontend MVP placeholders.
- Data is mainly based on mock/local data.

13. Conclusion

TastyBite is a frontend-only Restaurant App MVP developed using React Native and Expo. It provides the main customer and manager functionalities required for a restaurant application, including menu browsing, cart management, checkout, order tracking, reservations, profiles, and manager order management.